

16. Cross-Language Search Fallback

Phase: L **Code:** `search_api/services/language_router.py`, `search_api/services/rrf.py`, `search_api/services/search.py`

What it is

Cross-language search fallback is a retrieval strategy for bilingual systems where, if a search in the user's detected language doesn't return enough good results, the system automatically re-runs the keyword search in the *other* language and blends those secondary results into the final ranking at a reduced weight.

The key insight is that a bilingual database has two fundamentally different kinds of search index:

- **Vector (embedding) search** is language-agnostic. An embedding model maps "machine" and "gep" (Hungarian for "machine") to nearby points in the same vector space, so a single vector query already searches across both languages implicitly.
- **BM25 (keyword) search** is language-specific. BM25 works by matching lemmatized tokens — the Hungarian lemma "gep" will never match the English token "machine" because they're stored in separate tsvector columns (`bm25_tokens_hu` and `bm25_tokens_en`). A Hungarian query only hits the Hungarian column by default.

Cross-language fallback bridges this gap for BM25: when the primary-language keyword search comes up short, the system queries the other language's BM25 column too, so a Hungarian user searching for a concept that happens to be better described in English content can still find it — but those secondary-language results are deliberately downweighted (0.5x) so they don't outrank good same-language matches.

Real-life analogy

Imagine you walk into a bilingual library that has two card catalogues — one organized by Hungarian keywords, one by English keywords. The librarian speaks both languages.

You ask for books about "gepeszet" (mechanical engineering) in Hungarian.

1. **Step 1 — Language detection:** The librarian hears your question and thinks, "That's Hungarian."
2. **Step 2 — Primary search:** She checks the Hungarian card catalogue and finds 1 book. That's not many.
3. **Step 3 — Fallback decision:** She has a personal rule: "If I find fewer than 3 books in the primary catalogue, I should also check the other one." One book is less than three, so she walks over to the English card catalogue.
4. **Step 4 — Secondary search:** She searches the English catalogue for the same concept and finds 5 books about "mechanical engineering."

5. **Step 5 — Blending with reduced priority:** She puts all 6 books on your table, but places the Hungarian one on top and the English ones below, because you asked in Hungarian — the English results are *helpful additions*, not the primary answer.

The 0.5x weight is like the librarian's rule: "Books from the other catalogue are worth including, but they should sit behind same-language matches unless they're truly the only good results."

Why we used it here

Two requirements drive this design:

R14 (Bilingual RRF weighting) says that when fusing results from multiple sources, the secondary language's BM25 results must be weighted less than the primary language's. The spec doesn't prescribe an exact weight, but it's clear that same-language keyword matches should rank higher than cross-language ones, all else being equal. We use 0.5x — enough to surface useful cross-language results, but not enough to let a strong secondary-language keyword match leap over a good primary-language semantic match.

R19 (Cross-language fallback) says the system must be able to fall back to the other language when primary-language results are insufficient. The client requirement doesn't specify exactly what "insufficient" means (no threshold or minimum count is given in REQUIREMENTS.md §3, item 7), so we chose a sensible default: if fewer than 3 primary BM25 results score above 0.01, trigger the fallback. This is a conservative threshold — it avoids triggering on every query while ensuring the system doesn't return an empty or near-empty result set when perfectly good content exists in the other language.

Code walkthrough

1. Language detection and BM25 column routing — `search_api/services/language_router.py`

```
from langdetect import detect, DetectorFactory

DetectorFactory.seed = 0 # deterministic detection across runs

def detect_query_language(query: str) -> str:
    if not query or not query.strip():
        return "hu" # empty query defaults to Hungarian
    try:
        detected = detect(query)
    except Exception:
        return "hu" # detection failure defaults to Hungarian
    if detected == "hu":
        return "hu"
    if detected in ("en", "en-US", "en-GB"):
        return "en"
    return "hu" # any other language defaults to Hungarian
```

This function is the entry point for the entire language-routing decision. It uses the `langdetect` library (a port of Google's language detection) to classify the query. Three design choices worth noting:

- **Deterministic seed:** `DetectorFactory.seed = 0` ensures the same query always gets the same language classification, even though `langdetect` internally uses a probabilistic algorithm. Without this, the same query could occasionally flip between "hu" and "en" across runs.
- **Hungarian as default:** Since this is a Hungarian-primary system, any ambiguous or unrecognized language falls back to Hungarian. Short queries (1-2 words) are notoriously hard for language detectors to classify, so defaulting to the primary language is the safer choice.
- **Narrow English match:** Only "en", "en-US", and "en-GB" are treated as English — other Germanic languages that `langdetect` might return (like "de" for German) fall to the Hungarian default.

The column routing is a simple mapping:

```
def bm25_column(lang: str) -> str:
    return f"bm25_tokens_{lang}" # "hu" → "bm25_tokens_hu", "en" → "bm25_tokens_en"
```

This is why BM25 is language-specific: each language has its own `tsvector` column in the database, built from lemmas produced by the language's own NLP model (`huspacy` for Hungarian, `spaCy` for English). The `bm25_column()` function routes the query to the correct column based on the detected language.

2. The fallback decision — `search_api/services/language_router.py`

```
FALLBACK_SCORE_THRESHOLD = 0.01
FALLBACK_MIN_RESULTS = 3

def should_fallback(primary_results: list, score_threshold: float = FALLBACK_SCORE_THRESHOLD)
    -> bool:
    above = [r for r in primary_results if r.get("bm25_rank_score", 0) >= score_threshold]
    needs_fallback = len(above) < FALLBACK_MIN_RESULTS
    if needs_fallback:
        logger.info(
            f"EVIDENCE_API_CROSS_LANGUAGE_FALLBACK: "
            f"primary_above_threshold={len(above)} "
            f"min_required={FALLBACK_MIN_RESULTS} - triggering secondary-language search"
        )
    return needs_fallback
```

This function answers: "Did the primary-language BM25 search return enough good results, or should we also search the other language?"

- `score_threshold = 0.01`: PostgreSQL's `ts_rank()` returns a float indicating how well a document matches the query. Scores below 0.01 are essentially noise — the document barely matched a token. We only count results that are above this threshold.
- `min_results = 3`: If fewer than 3 primary results clear the threshold, the fallback fires. The number 3 is a sensible default for "too few" — one or two results suggests the primary language's index doesn't have strong coverage for this topic.
- `EVIDENCE_API_CROSS_LANGUAGE_FALLBACK` **log marker**: This is the UAT evidence trail (per CLAUDE.md's convention) — the log line proves, in production logs, that the fallback decision was made and why.

3. Secondary-language weight in RRF fusion — `search_api/services/rrf.py`

```
SECONDARY_LANGUAGE_WEIGHT = 0.5
```

```
def fuse_rankings(
    vector_results: list[str],
    bm25_primary_results: list[str],
    bm25_secondary_results: list[str] | None = None,
    k: int = DEFAULT_K,
) -> list[tuple[str, float]]:
    bm25_secondary_results = bm25_secondary_results or []
    scores: dict[str, float] = {}

    # Vector: full weight (1.0) – language-agnostic, always reliable
    for rank, item_id in enumerate(vector_results, start=1):
        scores[item_id] = scores.get(item_id, 0.0) + 1 / (k + rank)

    # BM25 primary: full weight (1.0) – same language as the query
    for rank, item_id in enumerate(bm25_primary_results, start=1):
        scores[item_id] = scores.get(item_id, 0.0) + 1 / (k + rank)

    # BM25 secondary: half weight (0.5) – other language, helpful but demoted
    for rank, item_id in enumerate(bm25_secondary_results, start=1):
        scores[item_id] = scores.get(item_id, 0.0) + SECONDARY_LANGUAGE_WEIGHT / (k + rank)

    fused = sorted(scores.items(), key=lambda pair: pair[1], reverse=True)
    return fused
```

The key line is `SECONDARY_LANGUAGE_WEIGHT / (k + rank)` instead of `1 / (k + rank)`. In standard RRF, every source contributes $1/(k+rank)$ to an item's score. By multiplying the secondary BM25 contribution by 0.5, we're saying: "A #1 result in the secondary language's BM25 is worth as much as a #2 result in the primary language's BM25." This prevents a scenario where, say, a strong English keyword match outranks a good Hungarian semantic match just because the user happened to use a term that translates well.

Why 0.5 and not some other number? It's a pragmatic middle ground:

- **1.0 (equal weight)** would treat secondary-language keyword matches as equally important as primary-language ones, defeating the purpose of language routing.
- **0.0 (zero weight)** would ignore secondary results entirely, defeating the purpose of fallback.
- **0.5** lets secondary results appear in the final ranking if they're genuinely relevant (high rank in their list), but they need to be roughly twice as highly ranked to compete with primary-language results.

4. The complete fallback flow — `search_api/services/search.py`

```
def search_table(
    table: Literal["category_vectors", "company_vectors"],
    query: str,
    limit: int = 20,
    lang_override: str | None = None,
    ...
) -> dict:
    # Step 1: Detect query language (or use override from request)
    detected_lang = lang_override or detect_query_language(query)

    # Step 2: Embed query (language-agnostic – same vector for any language)
    query_vec = embed_query(query)

    conn = get_connection()
    try:
        with conn.cursor(cursor_factory=RealDictCursor) as cur:
            # Step 3: Vector search – always searches the detected language's rows
```

```

vector_rows = _vector_search(
    cur, table, query_vec, detected_lang, limit, id_col, ...
)
# Step 4: BM25 primary — searches the detected language's tsvector column
bm25_primary_rows = _bm25_search(
    cur, table, query, detected_lang, limit, id_col, ...
)

vector_ids = [r[id_col] for r in vector_rows]
bm25_primary_ids = [r[id_col] for r in bm25_primary_rows]

# Step 5: Fallback decision — are primary BM25 results sufficient?
bm25_secondary_ids = []
secondary_lang = other_language(detected_lang)
bm25_primary_dicts = [dict(r) for r in bm25_primary_rows]
if should_fallback(bm25_primary_dicts):
    # Step 6: Secondary BM25 — search the OTHER language's column
    bm25_secondary_rows = _bm25_search(
        cur, table, query, secondary_lang, limit, id_col, ...
    )
    bm25_secondary_ids = [r[id_col] for r in bm25_secondary_rows]

# Step 7: Fuse all three sources (secondary list may be empty)
fused = fuse_rankings(vector_ids, bm25_primary_ids, bm25_secondary_ids)
finally:
    conn.close()

return {
    "detected_language": detected_lang,
    ...
    "cross_language_fallback_triggered": len(bm25_secondary_ids) > 0,
}

```

The flow reads top-to-bottom as a clear decision tree:

1. **Detect language** — determines which BM25 column to query first.
2. **Embed query** — produces a single vector that works across both languages (embeddings are language-agnostic).
3. **Vector search** — always runs, always at full weight. Searches only the detected language's rows, but the embeddings themselves capture cross-language meaning.
4. **BM25 primary** — searches the detected language's `bm25_tokens_{lang}` column.
5. **Fallback check** — `should_fallback()` counts how many primary BM25 results are above the score threshold. If fewer than 3, the fallback fires.
6. **BM25 secondary** (conditional) — if fallback triggered, searches the other language's BM25 column.
7. **RRF fusion** — merges all results. Vector and primary BM25 get full weight (1.0); secondary BM25 gets half weight (0.5). The response includes `cross_language_fallback_triggered: true/false` so the caller (and the UAT evidence logs) know whether fallback was used.

How to explain this in an interview/to a teammate

"Our search system is bilingual (Hungarian and English), and the two retrieval paths handle languages differently. Vector search is language-agnostic — embeddings map both languages into the same space — but BM25 keyword search is language-specific because Hungarian and English have completely different lemmatization rules stored in separate tsvector columns. So when a user searches in Hungarian,

we first query the Hungarian BM25 column; if we get fewer than 3 results above a minimum relevance threshold, we automatically fall back to the English BM25 column too. Those secondary-language results go into RRF fusion at half weight (0.5x) so they can surface useful cross-language matches without outranking good same-language results. The threshold and minimum count aren't specified by the client, so we chose conservative defaults that avoid triggering on every query while preventing empty result sets when relevant content exists in the other language."